

Dokumentacja Techniczna — Gym App

Aplikacja mobilna do zarządzania treningami siłowymi

Platforma: Flutter (Android / iOS) | Backend: Firebase (Auth + Firestore) | Wersja: 1.0.0+1

Spis treści

- Opis projektu
- Architektura i struktura katalogów
- Technonogie i zależności
- Modele danych
- Serwisy (warstwa logiki)
- Ekrany (warstwa UI)
- Schemat bazy danych Firebase
- Przepływ użytkownika
- Konfiguracja i uruchomienie

1. Opis projektu

Gym App to mobilna aplikacja Flutter wspomagająca trening siłowy na siłowni. Umożliwia użytkownikom:

- Rejestrację i logowanie przez e-mail (Firebase Auth)
- Tworzenie i edytowanie spersonalizowanych planów treningowych
- Przeprowadzanie treningu w czasie rzeczywistym ze stoperem
- Logowanie wykonanych serii (ciężar, powtórzenia, ukończenie)
- Przeglądanie historii wszystkich odbytych treningów

Dane każdego użytkownika są przechowywane w odizolowanej kolekcji Firestore przypisanej do jego konta (`users/{uid}/...`), co zapewnia pełną prywatność i bezpieczeństwo.

2. Architektura i struktura katalogów

Projekt stosuje uproszczoną architekturę warstwową (Screens → Services → Firebase):

```
lib/  
├── main.dart                # Punkt wejścia aplikacji  
├── firebase_options.dart    # Konfiguracja Firebase (auto-generowane)  
├──  
├── ekrany/  
│   ├── ekran_start.dart    # Ekran główny (dashboard)
```

```

├── ekran_login.dart           # Logowanie / rejestracja
├── ekran_tworzenie_planu.dart # Tworzenie nowego planu
├── ekran_edycji_planu.dart    # Edycja istniejącego planu
├── ekran_lista_planow.dart    # Lista zapisanych planów
├── ekran_aktywny_trening.dart # Aktywny trening ze stoperem
├── ekran_historia_treningow.dart # Historia odbytych treningów
├── models/                   # Modele danych
│   ├── exercise.dart        # Ćwiczenie (baza)
│   ├── workout_exercise.dart # Ćwiczenie w planie (z parametrami)
│   ├── workout_plan.dart     # Plan treningowy
│   └── workout_session.dart  # Sesja treningowa + zestaw + seria
├── services/                 # Warstwa logiki / dostępu do danych
│   ├── service_auth.dart     # Autentykacja Firebase
│   ├── plan_service.dart     # Zarządzanie planami (lokalne)
│   ├── session_service.dart  # Zapis/odczyt sesji z Firestore
│   └── database_service.dart  # Klasa szkieletowa

```

3. Technologie i zależności

Pakiet	Wersja	Zastosowanie
flutter SDK	—	Framework UI
firebase_core	^4.10.0	Inicjalizacja Firebase
firebase_auth	^6.5.2	Rejestracja, logowanie, wylogowanie
cloud_firestore	^6.5.0	Baza danych (plany, sesje)
cupertino_icons	^1.0.8	Ikony iOS

Wymagania środowiskowe:

- Flutter SDK $\geq 3.x$ (Dart SDK ^3.9.2)
- Android: compileSdk 35, minSdk per build.gradle
- iOS: Runner.xcodeproj skonfigurowany w Xcode
- Projekt Firebase z włączonymi usługami: Authentication (Email/Password) + Firestore

4. Modele danych

4.1 Exercise — lib/models/exercise.dart

Model reprezentujący pojedyncze ćwiczenie z bazy.

Pole	Typ	Opis
id	String	Unikalny identyfikator
name	String	Nazwa ćwiczenia (np. "Przysiady")
muscleGroup	String	Grupa mięśniowa (np. "Nogi")

4.2 WorkoutExercise — lib/models/workout_exercise.dart

Ćwiczenie jako element planu treningowego — zawiera docelowe parametry wykonania.

Pole	Typ	Opis
exerciseId	String	Odniesienie do Exercise.id
exerciseName	String	Nazwa (zduplikowana dla szybkości odczytu)
series	int	Liczba serii
powtorzenia	int	Liczba powtórzeń na serię
kg	double	Docelowy ciężar w kg

4.3 WorkoutPlan — lib/models/workout_plan.dart

Plan treningowy użytkownika, zapisywany w Firestore.

Pole	Typ	Opis
id	String	ID dokumentu Firestore (auto-generowane)
name	String	Nazwa planu (np. "Nogi", "Push A")
exercises	List<WorkoutExercise>	Lista ćwiczeń w planie

4.4 WorkoutSession, PerformedExercise, WorkoutSet — lib/models/workout_session.dart

Trzy klasy reprezentujące kompletny zapis sesji treningowej.

WorkoutSet — pojedyncza seria

Pole	Typ	Domyślnie	Opis
weight	double	—	Ciężar użyty w serii
reps	int	—	Liczba wykonanych powtórzeń
isCompleted	bool	false	Czy seria została ukończona

PerformedExercise — ćwiczenie wykonane podczas treningu

Pole	Typ	Opis
name	String	Nazwa ćwiczenia
sets	List<WorkoutSet>	Lista wykonanych serii

WorkoutSession — cała sesja treningowa

Pole	Typ	Opis
id	String	ID dokumentu Firestore
planName	String	Nazwa planu (lub "Trening bez planu")
date	DateTime	Data i godzina treningu
durationInSeconds	int	Całkowity czas trwania w sekundach
exercises	List<PerformedExercise>	Lista wykonanych ćwiczeń

Uwaga: Data przechowywana jest jako Timestamp Firebase. Odczyt obsługuje zarówno Timestamp, jak i String (fallback).

5. Serwisy (warstwa logiki)

5.1 AuthService — lib/services/service_auth.dart

Obsługuje operacje autentykacji przez Firebase Auth.

Metoda	Zwraca	Opis
login(email, password)	Future<UserCredential>	Logowanie e-mailem i hasłem
register(email, password)	Future<UserCredential>	Rejestracja nowego konta
logout()	Future<void>	Wylogowanie użytkownika
userStream	Stream<User?>	Strumień zmian stanu autentykacji

5.2 SessionService — lib/services/session_service.dart

Zarządza zapisem i odczytem sesji treningowych z Firestore.

Metoda	Zwraca	Opis
saveWorkoutSession(session)	Future<void>	Zapisuje sesję w users/{uid}/sessions
getWorkoutSessions()	Stream<List<WorkoutSession>>	Pobiera sesje malejąco po dacie

Jeśli użytkownik nie jest zalogowany, getWorkoutSessions() zwraca pusty Stream.value([]).

5.3 PlanService — lib/services/plan_service.dart

Singleton z listą planów przechowywaną w pamięci operacyjnej (nie w Firebase). Używany wyłącznie przez ekran edycji planu — tworzenie planów korzysta bezpośrednio z Firestore.

Metoda	Opis
dodajPlan(plan)	Dodaje plan do lokalnej listy
usunPlan(id)	Usuwa plan z lokalnej listy
edytujPlan(nowyPlan)	Zastępuje plan o danym id

5.4 DatabaseService — lib/services/database_service.dart

Klasa szkieletowa — niezaimplementowana. Nie jest aktualnie używana.

6. Ekran (warstwa UI)

6.1 Ekran logowania — ekran_login.dart

Klasa: LoginScreen

Ekran uwierzytelniania z przełącznikiem trybu logowanie / rejestracja.

Pola formularza: E-mail, Hasło (ukryte)

Zachowanie:

- isLogin = true → wyświetla "Zaloguj", wywołuje AuthService.login()
- isLogin = false → wyświetla "Zarejestruj się", wywołuje AuthService.register()
- W przypadku błędu wyświetla SnackBar z komunikatem
- loading = true podczas oczekiwania na odpowiedź Firebase

6.2 Ekran główny (dashboard) — ekran_start.dart

Klasa: StartowyEkran

Centralny hub nawigacyjny aplikacji po zalogowaniu.

Zawartość: Ikona i podtytuł "Twój Trener Personalny", Karty statystyk (aktualnie statyczne: 0 Treningów / 0 Planów / 0 Dni).

Cztery przyciski nawigacyjne:

Przycisk	Docelowy ekran	Kolor
Stwórz plan treningowy	EkranTworzeniaPlanu	Domyślny
Twoje plany treningowe	EkranListaPlanow	Outlined
Rozpocznij trening (bez planu)	EkranAktywnyTrening (pusty)	Niebieski
Historia treningów	EkranHistoriaTreningow	Pomarańczowy

6.3 Ekran tworzenia planu — ekran_tworzenie_planu.dart

Klasa: EkranTworzeniaPlanu

Formularz tworzenia nowego planu treningowego z zapisem do Firestore.

Dostępne ćwiczenia (hardcoded):

ID	Nazwa	Grupa mięśniowa
1	Przysiady	Nogi
2	Wykroki	Nogi
3	Martwy ciąg	Plecy
4	Podciąganie	Plecy
5	Hip Thrust	Pośladki
6	Odwodzenie na maszynie	Pośladki
7	Przywodzenie na maszynie	Uda
8	Wyciskanie sztangi	Klatka piersiowa
9	Pompki	Klatka piersiowa
10	Wiosłowanie sztangą	Plecy

Logika formularza: Każde ćwiczenie posiada CheckboxListTile. Po zaznaczeniu checkboxa ujawniają się pola: Serie (domyślnie 3), Powtórzenia (domyślnie 10), Kg (domyślnie 0). Walidacja: nazwa planu i ≥ 1 ćwiczenie muszą być podane.

Zapis: Firestore → users/{uid}/plans (nowe ID auto-generowane). Wskaźnik ładowania w przycisku podczas zapisu.

6.4 Ekran edycji planu — ekran_edycji_planu.dart

Klasa: EkranEdycjiPlanu

Parametr: WorkoutPlan plan

Identyczny formularz jak tworzenie planu, lecz: Inicjalizowany z istniejącymi danymi planu (plan.exercises), checkboxy i pola wypełnione bieżącymi wartościami. Zapis przez PlanService().edytujPlan() (in-memory — patrz uwaga w §5.3).

6.5 Ekran listy planów — ekran_lista_planow.dart

Klasa: EkranListaPlanow

Wyświetla plany zalogowanego użytkownika z Firestore.

Zachowanie: FutureBuilder pobiera users/{uid}/plans przy każdym renderze. Każdy plan wyświetlany jako Card z: Nazwą planu i liczbą ćwiczeń, Przyciskiem edycji (✍ niebieski) → EkranEdycjiPlanu, Przyciskiem usunięcia (🗑 czerwony) → usuwa dokument z Firestore + setState. Kliknięcie w kartę → przejście do EkranAktywnyTrening z danymi planu.

6.6 Ekran aktywnego treningu — ekran_aktywny_trening.dart

Klasa: EkranAktywnyTrening

Parametry opcjonalne: nazwaPlanu, poczatkoweCwiczeniaRaw

Główny ekran prowadzenia treningu w czasie rzeczywistym.

Funkcje: Stoper — odlicza sekundy od startu (Timer.periodic), wyświetla MM:SS. Lista ćwiczeń — inicjalizowana z planu lub pusta (trening ad-hoc). Dodawanie ćwiczeń — dialog z polem tekstowym.

Dodawanie serii — przycisk + pod każdym ćwiczeniem. Edycja serii — inline pola kg i powt, checkbox ukończenia.

Zapis treningu: Tworzy obiekt WorkoutSession z aktualnym stanem. Wywołuje

SessionService.saveWorkoutSession(). Po sukcesie wraca do poprzedniego ekranu (Navigator.pop). Blokuje przycisk podczas zapisywania (_trwaZapisywanie).

6.7 Ekran historii treningów — ekran_historia_treningow.dart

Klasa: EkranHistoriaTreningow

Wyświetla listę wszystkich odbytych sesji treningowych w czasie rzeczywistym.

Zachowanie: StreamBuilder nasłuchuje SessionService.getWorkoutSessions() (dane real-time). Obsługuje stany: ładowanie / błąd / pusta lista / dane. Każda sesja jako Card z: Zielona ikona check_circle, Nazwa planu, Data w formacie DD.MM.RRRR GG:MM, Czas trwania w formacie Xm Ys. onTap zarezerwowany (nie zaimplementowany — planowany ekran szczegółów).

7. Schemat bazy danych Firebase

Struktura Firestore:

```
Firestore
├── users/
│   └── {uid}/
│       Auth)
│       ├── plans/
│       │   └── {planId}/
│       │       ├── name: string
│       │       └── exercises: array [
│       │           {
│       │               exerciseId: string,
│       │               exerciseName: string,
│       │               series: number,
│       │               powtorzenia: number,
│       │               kg: number
│       │           }
│       │       ]
│       └── sessions/
│           └── {sessionId}/
│               ├── planName: string
│               ├── date: timestamp
│               ├── durationInSeconds: number
│               └── exercises: array [
```



```

    {
      name: string,
      sets: array [
        {
          weight: number,
          reps: number,
          isCompleted: boolean
        }
      ]
    }
  ]
}

```

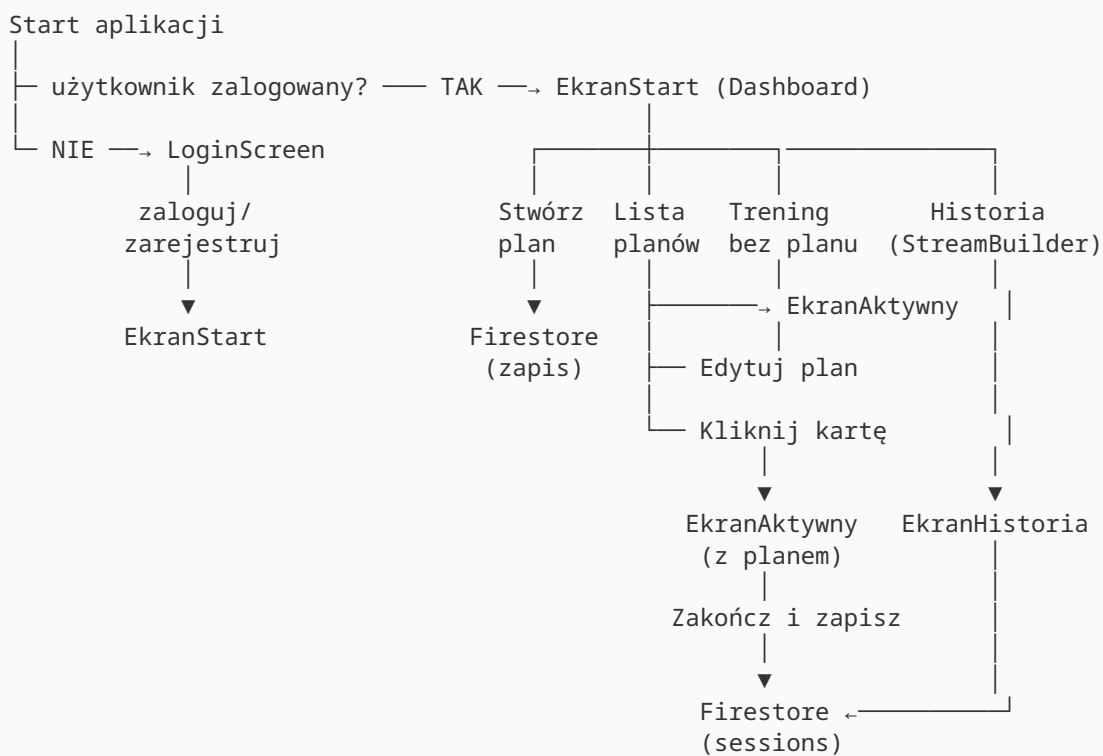
Reguły bezpieczeństwa (zalecane):

```

rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId}/{document=**} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }
  }
}

```

8. Przepływ użytkownika



9. Konfiguracja i uruchomienie

Wymagania wstępne:

1. Flutter SDK $\geq 3.x$ zainstalowany i w PATH
2. Konto Firebase z projektem (darmowy plan Spark wystarczy)
3. Włączone usługi Firebase: Authentication (Email/Password) + Cloud Firestore

Kroki konfiguracji:

```
# 1. Klonowanie / rozpakowanie projektu
cd gym_application_project-main

# 2. Instalacja zależności
flutter pub get

# 3. Konfiguracja Firebase
# Pobierz google-services.json (Android) z konsoli Firebase
# i umieść w: android/app/google-services.json
# (plik jest już obecny w projekcie – wymaga podmiany na własny)

# 4. Uruchomienie na podłączonym urządzeniu / emulatorze
flutter run

# 5. Budowanie APK (release)
flutter build apk --release
```